TYNDP-nahe Netzreduktion mit PyPSA

Implementierungsarchitektur mit minimaler Schrittzahl, mathematischen Modellen, API-Zuordnung und Abnahmekriterien

Technischer wissenschaftlicher Bericht

16. Oktober 2026

Zusammenfassung

Der Bericht leitet aus der ENTSO-E TYNDP-2026-Methodik zur Identification of System Needs (IoSN) einen möglichst kurzen und methodisch überprüfbaren Implementierungsweg mit PyPSA ab. Als einzige normative Quelle für die Bewertung der Netzreduktionsmethodik dient die ENTSO-E-Konsultationsfassung vom 25. Februar 2026. Die aktuelle PyPSA-Dokumentation und der Clustering-Quellcode dienen ausschließlich der Zuordnung tatsächlich vorhandener Softwarefunktionen. Vorgeschlagen werden fünf sequenzielle Arbeitsschritte: synchronisierte Referenzflüsse, engpassbewusstes Clustering, räumliche Aggregation, Kalibrierung elektrischer und sicherheitsbezogener Ersatzparameter sowie unabhängige Validierung. Die mathematischen Ergänzungen sind ausdrücklich Rekonstruktionen und keine veröffentlichten Originalalgorithmen der ENTSO-E. Die Implementierung kann mit PyPSA als zentralem Daten- und Modellframework erfolgen; drei schmale Zusatzmodule sind für TYNDP-nahe Treue notwendig.

Inhaltsverzeichnis

1 Executive Summary **2**

2 Wissenschaftliche Grundlage und Geltungsbereich **3**
2.1 Dokumentenbindung . . . 3
2.2 Ziel: Minimale Implementierungsarbeit statt minimaler physikalischer Prüfung . 4

3 Die fünf Verarbeitungsschritte **4**

4 Schritt 1: Datenharmonisierung und stündliche Referenzflüsse **5**
4.1 Eingangsvertrag und reproduzierbare Datenbasis [I] . . . 5
4.2 DC-Netzgleichungen [R] . . . 5
4.3 Priorisierung physikalischer Engpässe [D/R] . . . 5

5 Schritt 2: Engpassbewusstes, zusammenhängendes Clustering **6**
5.1 Knotenzuordnung und zwingende Eigenschaften [R] . . . 6
5.2 Bevorzugter PyPSA-Mechanismus [I] . . . 6
5.3 Minimal nötige Schutz- und Qualitätstests . . . 7

6 Schritt 3: PyPSA-Aggregation mit transparentem Physikvertrag **7**
6.1 Kernaufruf . . . 7
6.2 Vorsicht: Transformatoren und PST . . . 8
6.3 Bilanzexakte Aggregation [R] . . . 8
6.4 Was PyPSA-Standardaggregation nicht leisten kann . . . 9

7 Schritt 4a: Elektrische Kalibrierung der Ersatzreaktanzen **9**
7.1 Parameteridentifikation statt reiner Mittelung [D/R] . . . 9
7.2 Konkretes Kalibrierungsproblem [I] . . . 9
7.3 Minimaler Kalibrierungsaufwand und Zeitstichprobe . . . 10
7.4 Wichtiger PyPSA-Grenzfall: Flusskorrekturen im Dispatch . . . 10

8 Schritt 4b: N-1-äquivalente Transferkapazitäten 11
8.1 Was die Methode verlangt [D] . . . 11
8.2 BODF/LODF aus PyPSA [D aus Softwarequelle] . . . 11
8.3 Ersatzkapazität als konditionierte Grenzschätzung [I] . . . 12
8.4 Alternative Kontrollprüfung mit Transfer-Sensitivitäten [I] . . . 12
8.5 Vier Grenzwerte in PyPSA verwenden . . . 12

9 Sondergeräte, Zeitreihen und Einsatz in der Ausbauoptimierung 13
9.1 HVDC und PST: zuerst neutral kalibrieren, danach Wirkung isolieren . . . 13
9.2 Zeitreihen und Betriebsdaten erhalten . . . 13
9.3 LF- und OPF-Modus sind unterschiedliche Modellkonfigurationen . . . 14

10 Schritt 5: Unabhängige Validierung und wissenschaftliche Abnahme 14
10.1 Was tatsächlich nachgewiesen werden muss . . . 14
10.2 Qualitätsgrenzen: transparent und nicht fälschlich als Norm ausgeben . . . 15
10.3 Validierung an drei verschachtelten Netzeigenschaften . . . 15

11 API-Zuordnung und notwendiger Eigenanteil 16
11.1 Einfache Projektstruktur . . . 16
11.2 Minimaler ausrufbarer Ablauf . . . 17

12 Aufwand, Skalierung und bewusste Vereinfachungen 18
12.1 Drei Strategien im Vergleich . . . 18
12.2 Warum einige vermeintliche Abkürzungen falsch wären . . . 18
12.3 Sparsame Rechenstrategie [I] . . . 19
12.4 Grenzen der Nachweisbarkeit . . . 19

13 Schlussfolgerung und konkrete Architekturentscheidung 19

14 Literatur und dokumentierte Softwaregrundlagen 20

1 Executive Summary

Empfehlung. Die Netzreduktion sollte **nicht** durch einen neuen Power-Flow-Solver oder eine komplett neu geschriebene Clustering-Infrastruktur realisiert werden. Stattdessen bildet ein einzelnes PyPSA-`Network` den ausführbaren Datenkern; **fünf Verarbeitungsschritte** liefern aus einem detaillierten Modell ein belastbares zonales Netz. Die zentralen PyPSA-Werkzeuge sind `Network.lpf`, `SubNetwork.calculate_PTDF`, `SubNetwork.calculate_BODF`, `Network.cluster.spatial.busmap_by_hac` und `Network.cluster.spatial.get_clustering_from_busmap` [P1–P4].

Wichtigste Entscheidung: Die eigentliche zonale Topologie ist ein **explizit überprüfter `busmap`**. Sie wird engpassbewusst erstellt, einmal mit PyPSA in einen Netzgraphen überführt und danach **flussbasiert** kalibriert. Die standardmäßig aggregierten PyPSA-Leitungsreaktanzen und Grenzleistungen sind nur **Initialisierungswerte**, nicht die endgültigen ENTSO-E-Ersatzparameter. Diese Unterscheidung verhindert eine zentrale Fehlinterpretation. [E1, § 3.8.2 und Anhang 6.1]

Es gibt zwei definierte Ausbaustufen:

- **Basisreduktion (BR):** zeitabhängiger DC-Lastfluss, topologisch zusammenhängendes zonales HAC, integrierte PyPSA-Aggregation und stündliche Flussvalidierung. Sie ist schnell verfügbar, aber **nicht TYNDP-äquivalent**.
- **TYNDP-nahe Reduktion (TR):** BR plus geschützte Engpasskanten, optimierte Ersatzreaktanzen und strukturelle Nullbilanzflüsse, N-1-orientierte vier Kapazitäten pro Interzonenverbindung sowie korrekt modellierte PST-/HVDC-Einflüsse. Dies ist die **empfohlene wissenschaftliche Zielstufe**.

Reale Vorbedingungen: Ein ausreichend vollständiges AC-Übertragungsnetz, stündlich synchronisierte nodale Nettoeinspeisungen beziehungsweise ein konsistent zurückverteilter Marktdispatch, relevante Leitungsratings und definierte Kontingenzlisten. Sind diese Datensätze nicht vorhanden, kann PyPSA sie nicht allein aus der Topologie rekonstruieren. Die 102 Zonen der ENTSO-E beziehen sich auf ihr pan-europäisches Modell und sind kein universelles Optimum für jedes Eingangsnetz. [E1, § 3.8]

Explizite Softwaregrenzen: PyPSA-Standard-Clustering übernimmt `Line` und `Link` sowie 1-Port-Komponenten, aber **keine `Transformer`-Komponenten als solche**. Darüber hinaus hat die Standard-`Line`-Kapazitätsgrenze keine getrennten positiven und negativen saisonalen Flussgrenzen. Beides benötigt eine kontrollierte Anpassung vor bzw. nach dem Clustering. Die aktuelle PyPSA-API hat außerdem Einschränkungen beim räumlichen Clustering von bereits als Szenario-Netz markierten Objekten; Wetterfälle daher für die Reduktion extern getrennt führen. [P2–P5]

Stufe	Wesentliche Funktionen	Minimaler Zusatzaufwand	Als ENTSO-E-nah bewertbar?
BR	`lpf`, HAC, `get_clustering_from_busmap`	Daten- und Konsistenzprüfungen	Nein; nur Baseline
TR-Kern	BR, `scipy.optimize.least_squares`, `calculate_BODF`	drei fachliche Zusatzmodule	Ja, als **Rekonstruktion** bei bestandenen Tests
TR vollständig	TR-Kern, Linopy-Nebenbedingungen, PST/HVDC-Offsets	fokussierter Integrationsadapter	Ja, innerhalb nachgewiesener DC-Wirkungsgrenzen

2 Wissenschaftliche Grundlage und Geltungsbereich

2.1 Dokumentenbindung

Die **einzige Quelle für die fachlichen ENTSO-E-Anforderungen** ist [E1]: *TYNDP 2026 Identification of System Needs Methodology*, Konsultationsfassung vom 25. Februar 2026, insbesondere §§ 3.7–3.9 und Anhang 6.1. Die früher erstellte mathematische Bewertung *Netzreduktion im ENTSO-E TYNDP 2026* ist ein Arbeitsvorläufer und wird nicht als unabhängige normative Quelle behandelt. Die PyPSA-Quellen [P1–P5] und die offizielle PyPSA-Implementierung [P6] belegen ausschließlich Eigenschaften, Namen und Grenzen der Software.

Kennzeichnung: **[D]** ENTSO-E dokumentiert; **[R]** konsistente mathematische Rekonstruktion; **[I]** konkret vorgeschlagene Implementierungsentscheidung; **[U]** in der ENTSO-E-Quelle nicht eindeutig bestimmt. Weder [R] noch [I] darf als veröffentlichter ENTSO-E-Algorithmus bezeichnet werden.

Die relevante ENTSO-E-Ausgabe umfasst ausdrücklich sechs Größen [D]: (1) No-Load Flows, (2) reduzierte Maschengleichungen, (3) Wirkung der HVDC-Systeme auf No-Load-Flüsse, (4) vier N-1-Kapazitäten für Sommer/Winter und beide Richtungen, (5) Ersatzimpedanzen und (6) PST-Wirkung in MW. Eine alleinige `busmap` mit aggregierten `Lines` erfüllt nur einen Teil davon. [E1, § 3.8.3]

2.2 Ziel: Minimale Implementierungsarbeit statt minimaler physikalischer Prüfung

Ein einziger Aufruf `cluster_by_hac()` wäre zwar kürzer, kann die entscheidenden Vorgaben nicht garantieren. Die minimale **wissenschaftlich vertretbare** Prozesskette braucht deshalb fünf Schritte. Die engpassbewusste Partition und die Kalibrierung bleiben zwei verschiedene Probleme: Netzpartitionierung bestimmt **welche** physikalischen Leitungen als Zonengrenze sichtbar sind; Äquivalenzbildung bestimmt **welche Flüsse und Kapazitäten** diese Grenzverbindungen repräsentieren. Ein zusätzlicher globaler Alternativframework-Stack wird nicht empfohlen.

Nicht im Geltungsbereich: dynamische oder Kurzschlussäquivalenz, AC-Spannungs- und Blindleistungstreue, vollständige europäische Marktsimulation ohne Eingangsdaten, Netzplanung auf der Detailnetztopologie sowie identische Reproduktion unbekannter interner ENTSO-E-Quellalgorithmen.

3 Die fünf Verarbeitungsschritte

Schritt	Eingang	Einmaliger Hauptschritt	Ergebnis und harter Gate
1 – Referenzfälle	detailliertes PyPSA-Netz, synchroner Dispatch, Ratings	`lpf` und Engpass-/Flussanalyse	referenzierte stündliche Flüsse; Bilanz und Vorzeichen konsistent
2 – Busmap	Überlastungsenergie, Netzgraph, Zonenkontingent	HAC plus Engpass- und Konnektivitätsprüfung	vollständige, gültige `busmap`, vorrangige Engpässe interzonal
3 – Aggregation	`busmap`, vollständige Komponentenliste	`get_clustering_from_busmap()` plus Transformator-Preflight	reduzierter Netzgraph, Energiezeitreihen, `linemap`, korrekte Zuordnung
4 – Identifikation	Zielgrenzflüsse und N-1-Stresspunkte	kalibrierte x^{eq}, c^0, vier K pro Verbindung	numerisch stabile und identifizierbare Parameter mit Güteflag
5 – Validierung	zurückgehaltene Stunden und Ereignisse	Vergleich Detail- gegen Ersatznetz	dokumentierte Fehler und zulässiger Einsatzbereich; Export

Der Ablauf ist **einmalig pro Netzversion beziehungsweise Gültigkeitsbereich** durchzuführen. Eine Rücksprungschleife nach Schritt 5 ist nur bei definierten Qualitätsverletzungen zulässig; eine unbegrenzte heuristische Suche wird nicht vorgeschlagen. Für Zieljahre und Wetterzustände können gemeinsame Zonengrenzen mit je eigenständigen Kalibrierungsparametern untersucht werden. Ob das genau der internen ENTSO-E-Handhabung entspricht, bleibt [U].

4 Schritt 1: Datenharmonisierung und stündliche Referenzflüsse

4.1 Eingangsvertrag und reproduzierbare Datenbasis [I]

Benötigt werden mindestens:

1. **Netz:** eindeutige Bus- und Branch-IDs, AC-`Line` und `Transformer`, Spannungsebenen, Schalt-/Aktivstatus, Impedanzen, zulässige Belastungen, Ländernamen, geografische Koordinaten sowie HVDC-/PST-Kennzeichnung.
2. **Markt und Einspeisungen:** synchrone stündliche Lasten, Erzeugungs- und Speicherdispatch, Interkonnektor-Austausch sowie konsistente geografische und elektrische Anlagenzuordnungen. Reine Verfügbarkeitskurven sind kein Ersatz für tatsächlichen Dispatch.
3. **Kalibrierung:** Auswahl der Zieljahre, Wetter- und Betriebsfälle, relevante Leitungen (methodisch 225/400 kV), saisonale Zugehörigkeit, stündliche oder statische Ratings, geprüfte Ausfallkandidaten.
4. **Provenienz:** Version jedes Eingangssatzes und der PyPSA-/NumPy-/SciPy-Pakete; Zuordnung originaler zu aggregierten IDs; Hashes der Konfiguration.

Die TYNDP-Dokumentation behandelt Marktdispatch und PTDF-basiert rekonstruierte Detailnetzflüsse als getrennte Vorstufen. PyPSA übernimmt diesen Schritt nur, wenn die Dispatch-Zeitreihen in die wirksamen Knoteneinspeisungen umgesetzt werden. Ein `lpf()` mit beliebigen statischen Ausgangswerten ist dafür unzureichend. [E1, Anhang 6.1]

4.2 DC-Netzgleichungen [R]

Für Detailnetz-Inzidenz $A_d \in \{-1, 0, 1\}^{n \times m}$, Suszeptanzdiagonale $D_d = \mathrm{diag}(1/x_l)$ und Winkeln θ_t gilt in der verlustlosen Näherung:

$$A_d f_t^d = p_t^d, \qquad f_t^d = D_d A_d^{\mathsf{T}} \theta_t, \qquad A_d D_d A_d^{\mathsf{T}} \theta_t = p_t^d. \tag{1}$$

Erforderlich sind eine Slack-Winkelreferenz pro Zusammenhangskomponente, eine konsistente p.u.-Basis und eine bewusste Behandlung von Link-/PST-Sollwerten. Mit PyPSA entspricht das dem Grundprinzip von `n.lpf(snapshots=...)` beziehungsweise der Berechnung der PTDF pro Subnetz. `SubNetwork.calculate_PTDF()` und `SubNetwork.calculate_BODF()` erzeugen standardmäßig **dichte NumPy-Matrizen**; für große kontinentale Netze sind deshalb blockweises Rechnen und Kontingenz-Screening nötig. [P3]

Die von ENTSO-E verwendete PTDF-Referenzform kann als

$$f_{l,t}^d = f_{l,0}^d + \sum_{a \in \mathcal{A}} H_{la}(u_{a,t} - u_{a,0}) \tag{2}$$

formalisiert werden. [D/R] Die Menge $\mathcal{A}$ der Markt-Sensitivitätsparameter und die Referenzzustände sind nicht vollständig veröffentlicht [U]. Ist ein konsistenter vollständiger nodaler Dispatch vorhanden, kann (1) direkt gelöst werden; die finite-difference-PTDF-Prozedur muss dann **nicht zusätzlich** als zweiter Flussalgorithmus implementiert werden. Sie ist eine alternative Erzeugung der Referenzflüsse aus anderen verfügbaren Eingangsdaten.

4.3 Priorisierung physikalischer Engpässe [D/R]

Für die Originalleitung l im Wetterfall s:

$$E_{l,s}^{ov} = \sum_{t \in T_s} \Delta t_t \, \max\left(0, |f_{l,t,s}^{d}| - \overline{F}_{l,t,s}\right). \tag{3}$$

$E_{l,s}^{ov}$ besitzt bei MW und Stunden die Einheit MWh. Dies entspricht dem im Anhang beschriebenen Prinzip der jährlichen Überlastungsenergie, ohne unbekannte Schwellen oder Wettergewichtungen zu erfinden. Eine **mögliche** gewichtete Kombination mehrerer Szenarien $\sum_s \pi_s E_{l,s}^{ov}$ ist eine Entscheidung [I], keine definierte ENTSO-E-Vorschrift. Die prioritären Leitungen E_{crit} werden aus der Rangfolge und explizit dokumentierten Planungs-/Cross-Border-Vorgaben gebildet.

Gate 1: kein unaufgelöster Inselzustand; pro Stunde plausibler Gesamtausgleich; korrekte Zuordnung jeder kritischen Leitung, gegebenenfalls Übergang zu Netzkomponenten statt reiner `Line`-Tabelle; vollständiger Zeit- und Szenarioindex. Ausfälle, die Inselbildung verursachen, bleiben separat gekennzeichnet.

5 Schritt 2: Engpassbewusstes, zusammenhängendes Clustering

5.1 Knotenzuordnung und zwingende Eigenschaften [R]

Sei $P \in \{0,1\}^{k \times n}$ die Bus-Zonen-Matrix:

$$\sum_{z=1}^{k} P_{zi} = 1, \quad P_{zi} = 1 \Leftrightarrow g(i) = z. \tag{4}$$

Jede Zone muss innerhalb des berücksichtigten **passiven AC-Graphen** zusammenhängend sein. Die durch HVDC-`Link` erzeugte steuerbare Verbindung darf nicht als elektrische AC-Nachbarschaft interpretiert werden. Zusätzliche harte Kriterien sind vorab vergebene Länder-/Marktregiongrenzen, soweit diese für die konkrete Studie verbindlich sind. Die methodischen Forderungen nach kritischen Interzonenleitungen und ähnlichen Stromflussrichtungen werden durch zusätzliche Tests operationalisiert. [E1, §§ 3.8.1–3.8.2]

Mit E_{crit} als prioritären physischen AC-Kanten lautet die Schutzbedingung

$$P_{g(i),i} = P_{g(j),j} = 1 \text{ mit } g(i) \neq g(j) \qquad \forall\, l = (i,j) \in E_{crit}. \tag{5}$$

Dies ist eine vorgeschlagene **harte Variante** des ENTSO-E-Kriteriums [I]. Die ENTSO-E formuliert es als Clustering-Ziel und veröffentlicht keinen mathematischen Unbedingtzwang. Bei zu vielen kritischen Kanten kann die Bedingung mit einem vorgegebenen k **unlösbar** werden: Der Solver darf dann nicht heimlich das Zonenziel oder den Schutzstatus ändern, sondern muss priorisierte Ausnahmen oder eine höhere Zonenzahl berichten.

5.2 Bevorzugter PyPSA-Mechanismus [I]

PyPSA liefert eine graphenbewusste hierarchische Clusterbildung:

```
# Illustrative PyPSA-API; noch keine vollständige TYNDP-Busmap:
feature = make_bus_features(n, hours)   # eigene, busindizierte Merkmale
busmap0 = n.cluster.spatial.busmap_by_hac(
    n_clusters=k,
    buses_i=ac_bus_index,
```

```
    branch_components=["Line", "Transformer"],
    feature=feature,
    linkage="ward",
)
busmap = check_and_repair_busmap(
    n, busmap0, critical_edges, zone_limits
)  # eigene begrenzte Prüfroutine
```

`make_bus_features` und `check_and_repair_busmap` sind **zu implementierende** Funktionen, keine PyPSA-APIs. HAC ist gegenüber geographischem K-Means vorteilhaft, weil die passive Netzkonnektivität als Nachbarschaft berücksichtigt werden kann. **Wichtig:** HAC garantiert nicht automatisch, dass die Endpunkte *jeder* prioritären Leitung in verschiedenen Zonen liegen. Eine nachgelagerte Prüfung oder explizite Cannot-Link-Restriktion bleibt notwendig. [P1, P2]

Für ein vollständiges pan-europäisches Referenzmodell sind die ENTSO-E-Zonenbudgets aus [E1, Tabelle 3] zu übernehmen und die HAC-Bildung innerhalb der zulässigen Teilräume auszuführen. Die Zahl 102 wird nicht aus einem einzelnen europaweiten K-Means-Lauf ohne Landesrestriktionen nachgebildet. Busmerkmale können normierte Auslastungs-/Einspeiseprofile, Nähe zu kritischen Zweigen und optional elektrische Sensitivitäten enthalten. Die Gewichtungen dieser Merkmale sind [I], nicht [D].

5.3 Minimal nötige Schutz- und Qualitätstests

Nach jedem Clustering werden nur vier Indikatoren berechnet: (i) Vollständigkeit von g, (ii) Zusammenhang jeder Zone, (iii) Anteil priorisierter kritischer Zweige an Zonengrenzen und (iv) starke Vorzeichenkompensation aggregierter Grenzflüsse. Für das vierte Kriterium kann beispielsweise

$$\kappa_e = \frac{\sum_t \left| \sum_{l \in E_e} s_l f_{l,t} \right|}{\sum_t \sum_{l \in E_e} |f_{l,t}| + \epsilon} \in [0,1] \tag{6}$$

dienen. Ein kleiner Wert weist auf Flusskompensation hin; das ist ein **ergänzender** Kennwert [I]. Die Auswahl eines Grenzwertes für κ_e bleibt konfigurierbar. Die Routine repariert nur auffällige Zonen. Falls sie kein zulässiges Ergebnis findet, endet Schritt 2 mit einem Qualitätsfehler und **keiner** stillschweigenden Freigabe.

6 Schritt 3: PyPSA-Aggregation mit transparentem Physikvertrag

6.1 Kernaufruf

Nach bestandener Partitionierung übernimmt PyPSA die Bildung des reduzierten Datenmodells:

```
cluster = n.cluster.spatial.get_clustering_from_busmap(
    busmap,
    with_time=True,
    aggregate_one_ports={"Load": {}},
    scale_link_capital_costs=False,
)
nz = cluster.n
```

```
mapping_buses = cluster.busmap
mapping_lines = cluster.linemap
```

Die Syntax orientiert sich an den dokumentierten PyPSA-APIs. Der Lauf wird auf einem **normalen, nicht bereits in PyPSA-Szenarien aufgefächerten `Network`** ausgeführt. `Load` wird zur zonalen Leistungssumme zusammengeführt; einzelne Kraftwerke und Speicher bleiben zunächst als Assets erhalten, erhalten aber ihre zonale Bus-Zuordnung. Das vermeidet den Verlust von Kraftwerks-/Speichersemantik, der durch voreiliges Zusammenfassen pro Energieträger auftreten kann. Nach erfolgreicher Validierung darf eine weitere rein ökonomische Assets-Aggregation geprüft werden. [P1, P2, P6]

6.2 Vorsicht: Transformatoren und PST

Harter Implementierungsbefund aus dem PyPSA-Quellcode: `get_clustering_from_busmap` baut das reduzierte Netz explizit aus aggregierten `Bus` und `Line`, überträgt 1-Port-Komponenten und `Link`, fügt aber `Transformer` nicht analog wieder hinzu. Man darf deshalb **nicht** annehmen, dass ein Eingangsnetz mit Transformatoren nach diesem Aufruf elektrisch vollständig geblieben ist. [P6]

Die kleinstmögliche robuste Lösung ist ein kontrollierter Vor-/Nachbearbeitungsschritt:

- Für **passive, nicht regelbare Transformatoren** wird vorab festgelegt, ob ihre DC-äquivalente Reaktanz in die spätere äquivalente AC-Verbindung einfließt; ihre elektrische Wirkung und die Ursprungs-IDs sind dabei zu erhalten. Ein bloßes Kopieren von `x` ohne Tap-, Spannungsbasis- und p.u.-Konversion ist unzulässig.
- **PSTs** werden separat gekennzeichnet. Bleiben sie zwischen Zonen erhalten, können sie als `Transformer` mit Phasenverschiebung modelliert werden; innerhalb einer Zone verschwundene PSTs erfordern eine gesonderte äquivalente Flusswirkungsbeschreibung oder eine Zonenänderung.
- Für jede zusammengefasste physikalische AC-Verbindung wird eine Herkunftstabelle erzeugt, die Line/Transformer-Typ, Orientierung, Spannungsebene und Parameterbasis enthält. Die Summe der Flüsse auf allen ursprünglichen Grenzkomponenten wird später **nicht** aus `cluster.linemap` allein ohne Vorzeichenprüfung abgeleitet.

Alternativ kann ein vorab geprüftes rein passives AC-Netz mit einheitlich in `Line` abgebildeten Zweigen als Eingangsnetz verwendet werden; dann reduziert sich der Zusatzcode. Diese Vereinfachung ist nur zulässig, wenn dabei weder PST-Wirkung noch für die Studie relevante Transformatorgrenzen verloren gehen.

6.3 Bilanzexakte Aggregation [R]

Die ursprünglichen Grenzzweigflüsse werden mit der orientierten Matrix $\Gamma \in \{-1, 0, 1\}^{r \times m}$ auf Interzonenverbindungen e abgebildet:

$$f_t^\star = \Gamma f_t^d, \qquad q_t = P p_t^d. \tag{7}$$

Orientiert man Original- und Zielkanten konsistent, folgt die **strukturelle Identität**

$$\boxed{A_z \Gamma = P A_d.} \tag{8}$$

Aus $A_d f_t^d = p_t^d$ folgt damit

$$A_z f_t^\star = q_t. \tag{9}$$

Diese Identität liefert den wichtigsten Vorabtest des Aggregationscodes: Zonale Ein- und Ausspeisung **muss** bei gleichem Originaldispatch exakt (bis auf numerische Rundung) bilanziert sein. Sie beweist aber **nicht**, dass die Ersatznetzmaschengleichungen oder die N-1-Belastungen exakt übereinstimmen.

6.4 Was PyPSA-Standardaggregation nicht leisten kann

Die PyPSA-Vorgabestrategien aggregieren `Line.x` unter anderem mittels eines reziprok-spannungsgewichteten Durchschnitts und `Line.s_nom` über eine Summe [P6]. Das ist als netztechnische Anfangsnäherung möglich, entspricht aber **nicht** der ENTSO-E-Anforderung einer Kalibrierung gegen Zielflussstunden und einer N-1-orientierten Kapazitätsschätzung. Auch das neue geometrische `length` darf nicht mit der physikalischen Leitungsstreckenlänge verwechselt werden, sofern sie aus Zonenmittelpunkten entsteht.

Gate 3: vollständige Komponentenliste; kein stiller Verlust grenzüberschreitender Transformatoren oder PSTs; $A_z\Gamma = PA_d$ geprüft; statische Kapazitäten und stündliche Energie-/Dispatch-Zeitreihen erhalten; alle Zuordnungen versioniert.

7 Schritt 4a: Elektrische Kalibrierung der Ersatzreaktanzen

7.1 Parameteridentifikation statt reiner Mittelung [D/R]

Sei $A_z \in \mathbb{R}^{k\times r}$ die orientierte Inzidenz des reduzierten Netzes, $X = \mathrm{diag}(x_e^{eq})$ die positive Reaktanzmatrix und $C \in \mathbb{R}^{r\times(r-k+c)}$ eine Zyklenmatrix mit $A_zC = 0$ (c: Anzahl zusammenhängender Teilnetze). Die physikalisch richtige Zielgröße ist der anhand von (7) berechnete **aggregierte Detailnetzfluss** $f_t^\star$, nicht der von der Standard-PyPSA-Aggregation zufällig gelieferte Fluss.

Für die kalibrierte DC-Struktur wird ein struktureller Nullbilanzfluss c^0 eingeführt:

$$A_z\hat{f}_t = q_t, \qquad A_zc^0 = 0, \qquad C^\mathsf{T}X(\hat{f}_t - c^0) = 0. \tag{10}$$

Ein physikalisch konsistenter algebraischer Ansatz lautet

$$c^0 = C\eta, \qquad \hat{f}_t = X^{-1}A_z^\mathsf{T}\hat{\theta}_t + C\eta. \tag{11}$$

Dann gilt $A_zc^0 = 0$ konstruktionsbedingt. Dies ist eine **Rekonstruktion** der veröffentlichten Idee von Ersatzimpedanzen und No-Load Flows, nicht die von ENTSO-E veröffentlichte interne Parametrisierung. [E1, §§ 3.8.2–3.8.3, Anhang 6.1]

Die Rolle von c^0 ist wesentlich: Sind die zonalen Nettoaustausche null, kann das detaillierte Netz aufgrund verteilter Einspeisungen, Lasten und Ringflüsse dennoch Interzonenflüsse aufweisen. Ohne c^0 erzwingt ein gewöhnliches verlustloses passives Ersatznetz für $q = 0$ und festen Winkeln in jedem zusammenhängenden Subnetz $f = 0$; dies kann die Zielgröße verfehlen. Ein rein baumförmiges Ersatznetz besitzt keinen Freiheitsgrad für diverganzfreie Loop-Flows; notwendige No-Load-Flüsse wären dann strukturell nicht identifizierbar.

7.2 Konkretes Kalibrierungsproblem [I]

Als möglichst kleines, reproduzierbares mathematisches Verfahren wird folgende **regularisierte nichtlineare kleinste-Quadrate-Schätzung** vorgeschlagen:

$$r_t(\xi, \eta) = W \left[\hat{f}_t(e^{\xi} \odot x^{(0)}, C\eta) - f_t^{\star} \right]. \tag{12a}$$

$$J(\xi, \eta) = \sum_{t \in T_{cal}} w_t \|r_t(\xi, \eta)\|_2^2 + \lambda_x \|\xi\|_2^2 + \lambda_c \|C\eta\|_2^2. \tag{12b}$$

$$\min_{\xi, \eta} J(\xi, \eta), \qquad \sum_e \omega_e \xi_e = 0, \quad \xi_{min} \leq \xi \leq \xi_{max}. \tag{12c}$$

Hier ist $x^{(0)}$ die nach kontrollierter PyPSA-Aggregation erhaltene Startreaktanz; $\xi_e = \log(x_e^{eq}/x_e^{(0)})$ erzwingt positive Reaktanzen; W und w_t erlauben eine explizite Gewichtung relevanter Verbindungen und Stunden. Die Nullsumme/Normierung der ξ adressiert die gemeinsame Skalierungsfreiheit der DC-Reaktanzen. λ_x und λ_c schützen vor instabilen oder unphysikalischen Kalibrierungen. Für jeden Kandidatenparametervektor wird eine kleine lineare DC-Gleichung des reduzierten Netzes gelöst; nötig ist **kein** neues detailliertes AC-Modell.

Für die tatsächlich geschätzten x^{eq} müssen die **PyPSA-konformen Einheiten** wiederhergestellt werden. `Line.x` und die abgeleitete Größe `x_pu_eff` sind nicht dasselbe Eingabefeld. Transformatorreaktanzen besitzen zudem andere Datenbasen und mögliche Tap-Anpassungen. Die explizite Funktion `calculate_dependent_values()` und eine numerische Gleichheitsprüfung des Suszeptanzoperators dienen als Konvertierungs-Gate. [P3, P5]

Keine Überinterpretation: ENTSO-E dokumentiert die Minimierung des Flussfehlers unter Verwendung von Ersatzimpedanzen und Korrekturen, veröffentlicht aber weder (12a–12c) noch dessen Gewichte, Parameternormierung oder Stundenauswahl. Entsprechend müssen diese Festlegungen im Bericht der konkreten Implementierung als [I] protokolliert werden. [E1, S. 27 und 48–49]

7.3 Minimaler Kalibrierungsaufwand und Zeitstichprobe

Für die Optimierung ist nicht zwingend jeder Stundenwert notwendig. Der Vorschlag [I] verwendet eine **deterministische, szenario-stratifizierte Teilmenge**: (a) repräsentative Last-/Einspeisemuster, (b) Stunden mit den größten aggregierten Flüssen, (c) Stunden mit hoher Detailnetz-Überlastungsenergie und (d) Fälle mit besonderen HVDC-/PST-Einflüssen. Die tatsächlich aus [E1] nicht bekannte ENTSO-E-Auswahl wird damit nicht behauptet.

Sämtliche zur Kalibrierung **nicht** verwendeten Stunden bleiben für den Validierungsschritt reserviert; außerdem wird ein kompletter zurückgehaltener Wetterfall als Robustheitstest empfohlen. Die Stunde muss für alle Leitungen und Netze synchron sein: Aus verschiedenen Zeitpunkten zusammenmontierte Flussvektoren verletzen die Bilanz- und Maschengleichungen.

7.4 Wichtiger PyPSA-Grenzfall: Flusskorrekturen im Dispatch

Die PyPSA-Standard-KVL verwendet sinngemäß

$$C^{\mathsf{T}} X f_t = 0. \tag{13}$$

Für das oben rekonstruierte Modell wird dagegen

$$C^{\mathsf{T}} X f_t = C^{\mathsf{T}} X c^0 \tag{14}$$

benötigt. **Es reicht nicht**, nach einem unveränderten PyPSA-Lastfluss c^0 auf die Ergebnistabelle zu addieren, während die Dispatchoptimierung weiterhin (13) und unveränderte Leitungsgrenzen verwendet. Das würde zwar grafisch korrigierte Flüsse erzeugen, aber ein **anderes Optimierungsmodell** verwenden.

Für vollständige TR-Treue wird daher ein kompakter Linopy-Adapter vorgeschlagen: `n.optimize.create_model()`; KVL-Nebenbedingungen **kontrolliert** durch ihre affine Variante (14) ersetzen und nachprüfen, dass bestehende PST-Konstanten und weitere passive AC-Zweige korrekt berücksichtigt bleiben; danach `n.optimize.solve_model()`. PyPSA dokumentiert die Änderung der KVL-Bedingung im erstellten Modell als zulässigen Anpassungspunkt [P4]. Die tatsächliche Linopy-Dimensionsbehandlung ist versionsabhängig und vor einer produktiven Nutzung gegen einen vermaschten kleinen Prüffall zu testen. Für die **Offline-Validierung** der Kalibrierung ist ein kleiner eigener sparse DC-Löser ausreichend; ein unverändertes `nz.lpf()` berücksichtigt c^0 nicht.

8 Schritt 4b: N-1-äquivalente Transferkapazitäten

8.1 Was die Methode verlangt [D]

Für jede äquivalente Interzonenverbindung verlangt [E1] vier Kapazitäten, zwei Jahreszeiten mal zwei Transportrichtungen. Sie werden aus stündlich beobachteten Gesamtgrenzflüssen und dem **höchsten** kritischen Leitungsbelastungsgrad unter N-1-Sicherheitsbetrachtung abgeleitet. Weder die bloße Summe thermischer Leistungen noch ein pauschaler Abschlag wie 70 % erfüllt diese Vorgabe. Die ENTSO-E benennt außerdem problematische Fälle einseitiger Flüsse, fehlender Daten nahe 100 % und stark streuender Datenwolken. [E1, §§ 3.8.2–3.8.3, Anhang 6.1]

8.2 BODF/LODF aus PyPSA [D aus Softwarequelle]

Für physikalische Leitung l und Ausfall o gilt in linearer DC-Näherung

$$f_{l,t}^{(o)} = f_{l,t}^{d} + BODF_{lo}\, f_{o,t}^{d}. \tag{15}$$

PyPSA nennt diese Größe `BODF` (Branch Outage Distribution Factor), fachlich entspricht dies hier dem üblichen LODF-Konzept. `sn.calculate_BODF()` ermittelt eine **dichte** Matrix über die passiven Zweige eines zusammenhängenden Subnetzes und setzt auf der Diagonale $BODF_{oo} = -1$ [P3]. Für einen Ausfall, der das betrachtete AC-Teilnetz inselbildend trennt, ist die normale Sensitivitätsformel nicht ohne Weiteres gültig. Solche Fälle sind gesondert zu klassifizieren und dürfen nicht als sicher eingestuft werden, indem ein numerischer `NaN` ignoriert wird.

Für eine definierte Menge relevanter ausfallbelasteter Zweige $\mathcal{M}_e$ und Ausfallkandidaten $\mathcal{O}_e$ definiert der vorgeschlagene Kapazitätsindikator

$$R_{e,t} = \max_{\substack{l\in\mathcal{M}_e \\ o\in\mathcal{O}_e}} \frac{|f_{l,t}^{d} + BODF_{lo} f_{o,t}^{d}|}{\overline{F}_{l,t}}, \qquad F_{e,t} = \sum_{l\in E_e} s_l f_{l,t}^{d}. \tag{16}$$

Dies ist eine rekonstruierte **Datenpunktdefinition** [R/I] zur Umsetzung der ENTSO-E-Beschreibung. Die exakte CBCO-Auswahl und gegebenenfalls die Nichtberücksichtigung operativ beherrschbarer Ausfälle müssen als Eingabedaten begründet werden; sie lassen sich nicht allein aus der PyPSA-API ableiten.

8.3 Ersatzkapazität als konditionierte Grenzschätzung [I]

Für jeden Saison- und Richtungsindex (σ, d) wird die Datenmenge

$$\mathcal{D}_{e,\sigma,d} = \{(dF_{e,t}, R_{e,t}) \mid t \in T_\sigma,\ dF_{e,t} > 0\}, \quad d \in \{-1, +1\} \tag{17}$$

gebildet. Eine bewusst einfache reproduzierbare Standardoption ist eine robuste lineare Schätzung $R \approx a_{e,\sigma,d} + b_{e,\sigma,d}(dF)$ mit $b > 0$ und anschließend

$$K_{e,\sigma,d} = \frac{1 - a_{e,\sigma,d}}{b_{e,\sigma,d}}. \tag{18}$$

(18) ist **nicht** als veröffentlichter ENTSO-E-Fit zu interpretieren. Die Quelle legt die allgemeine Regressionsfunktion nicht eindeutig fest. Der Fit ist nur verwendbar, wenn genügend Daten, ein plausibler positiver Anstieg, eine belastbare Extrapolationsdistanz und unabhängige Kontingenzvalidierung vorliegen. Andere Verfahren, beispielsweise konservative Quantilregression, sind zulässig, müssen aber getrennt gekennzeichnet werden.

Der Bericht übernimmt die von [E1, Anhang 6.1] ausdrücklich beschriebenen Sonderfallentscheidungen in den Implementierungsvertrag: überwiegend einseitige Flüsse (**mehr als 99,5 %** der Zeit) können für die Gegenrichtung eine als solche gekennzeichnete **gespiegelte Ersatzannahme** erfordern; bei unzureichenden Beobachtungen über **1 %** Belastung wird kein belastbarer Wert erzeugt; bei über **99,5 %** der Beobachtungen über **130 %** nennt die Quelle einen gesonderten Fit vom Ursprung durch den Mittelwert. Stark gestreute Daten verlangen eine Warnung und gegebenenfalls eine Überarbeitung der Zonengrenze. Kein fehlender Wert wird stillschweigend mit null oder unendlich ersetzt.

8.4 Alternative Kontrollprüfung mit Transfer-Sensitivitäten [I]

Zur unabhängigen Prüfung kann ein inkrementeller Zonen-Transfer δ mit explizitem nodalem Verteilungsschlüssel h_e erzeugt werden:

$$f_{l,t}^{(o)}(\delta) = f_{l,t}^{(o)}(0) + (H_l h_e + BODF_{lo} H_o h_e)\,\delta. \tag{19}$$

Aus den linearen thermischen Ungleichungen $|f_{l,t}^{(o)}(\delta)| \le \overline{F}_l$ folgt ein zustandsabhängiges zulässiges Transferintervall. Dieses Verfahren ist nicht mit dem historisch beobachteten Fit identisch; es benötigt Verteilungsschlüssel der Erzeugungs- und Laständerungen. Sein Nutzen liegt in der **Gegenprüfung** der aus (18) abgeleiteten Transfergrenze unter nachvollziehbaren räumlichen Einspeisemustern. Die gleichzeitige Durchsetzbarkeit aller $K_{e,\sigma,d}$ für beliebige kombinierte Transfers ist damit noch nicht garantiert.

8.5 Vier Grenzwerte in PyPSA verwenden

PyPSA kennt `Line.s_nom` und einen (auch zeitabhängigen) Faktor `Line.s_max_pu` für **symmetrische** Betragsgrenzen. Die TYNDP-Werte sind jedoch richtungsabhängig, weshalb $K_{e,s}^{\text{pos}} \neq K_{e,s}^{\text{neg}}$ sein kann (mit Saisonindex s). Der korrekte Vertrag für den gesamten interzonalen Fluss lautet

$$-K_{e,s}^{\text{neg}} \le f_{e,t}^{\text{total}} \le K_{e,s}^{\text{pos}}, \qquad s = s(t). \tag{20}$$

Dies wird in der TYNDP-nahen Ausbaustufe über **benutzerdefinierte Linopy-Ungleichungen** ergänzt. Die symmetrischen PyPSA-Standardgrenzen müssen entsprechend entkoppelt

beziehungsweise so weit gesetzt werden, dass sie die asymmetrischen Werte nicht unbeabsichtigt verschärfen. Bei $c^0 \neq 0$ ist in (20) der **gesamte** Fluss f^{total} einschließlich struktureller Flusskorrektur zu begrenzen. Eine bloße Ersetzung durch das Minimum beider Richtungskapazitäten wäre sicherheitsorientiert, aber grundsätzlich **nicht gleichwertig** und kann die TYNDP-Transfermöglichkeiten systematisch unterschätzen.

Gate 4: gültige Reaktanzen, unabhängiger Flussfehler, Support-/Fitdiagnose jeder der vier Kapazitäten, dokumentierte geprüfte Ausfälle und Ausnahmefälle, korrekter Gleichungsimport in PyPSA oder explizite Deklaration eines nur offline kalibrierten Modells.

9 Sondergeräte, Zeitreihen und Einsatz in der Ausbauoptimierung

9.1 HVDC und PST: zuerst neutral kalibrieren, danach Wirkung isolieren

ENTSO-E setzt für eine robustere Ermittlung der Ersatzimpedanzen HVDC-Sollwerte im AC-Verbund auf null und PSTs auf die neutrale Stufe. Zusätzlich gehören die Wirkungen dieser Elemente zu den veröffentlichten Reduktionsoutputs. [E1, S. 49, § 3.8.3]

Die minimale mathematische Abbildung ist eine **Baseline plus separat identifizierte Wirkung**:

$$\hat{f}_t^{total} = \hat{f}_t(q_t; x^{eq}, c^0) + S_H \, \Delta h_t + S_P \, \Delta \varphi_t, \tag{21}$$

wobei die inkrementellen Beiträge nur unter einer **klar deklarierten Einspeise- und Gerätedefinition** addiert werden dürfen; andernfalls droht Doppelzählung gegenüber q_t. Besonders bei HVDC gilt: Der Link erzeugt bereits entsprechende Ein- und Ausspeisungen an seinen AC-Endknoten. Wird sein Beitrag so in q_t erfasst, darf $S_H \Delta h$ nicht nochmals voll addiert werden. Praktisch werden die AC-Flusswirkungen über denselben PTDF-Bezug berechnet und als sensitivitätsbasierte Nachweise gespeichert. PST-Wirkung kann über einen kleinen Phasenwinkel-Parameter-Sweep aus `lpf` oder einer entsprechenden linearen Perturbation bestimmt werden. Die ENTSO-E gibt keine vollständigen Sensitivitätsgleichungen vor [U].

9.2 Zeitreihen und Betriebsdaten erhalten

Die einfachste belastbare Wahl ist **räumliche** Aggregation bei unveränderter stündlicher Zeitachse. Deshalb `with_time=True`; ein temporales `resample` oder `segment` darf nicht gleichzeitig unkontrolliert angewendet werden, weil seltene Engpässe und die Kapazitätskalibrierung davon abhängen. Bei erforderlicher zeitlicher Verdichtung werden erst nach der räumlichen Validierung relevante Extremstunden explizit geschützt.

- **Last:** stündliche zonale Summe; Länder-/Regionalschlüssel als provenancehaltige Eingangsdaten.
- **Generatoren:** behalten zunächst individuelle Identitäten; `bus` zeigt auf die neue Zone. Verfügbarkeitszeitreihen und Dispatchzeitreihen werden separat geprüft.
- **Speicher:** individuelle Speicherzustände und Energiegrenzen zunächst beibehalten; naive Summation von Wirkungsgraden oder `max_hours` ist nicht exakt äquivalent.
- **Wetterprofile:** elektrische Bus-Zuordnung und meteorologische PECD-Region getrennt speichern; die elektrische Zuordnung bestimmt `busmap`, die meteorologische den zeitlichen Erzeugungsfaktor.

- **Szenarien:** Datensätze und Gewichte außerhalb des räumlichen Clusteraufrufs versionieren; nicht annehmen, dass die räumliche API Szenario-Multiindices unverändert unterstützt. [P2, P6]

9.3 LF- und OPF-Modus sind unterschiedliche Modellkonfigurationen

ENTSO-E beschreibt eine Ausbauoptimierung im LF-Modus, in der bestehende interne Engpässe das grenzüberschreitende Ausbauportfolio nicht begrenzen sollen, und nachgelagerte OPF-/Dispatch-Untersuchungen mit aktivierten internen Restriktionen. Der Quellentext ist bei der Verwendung des Wortes *interzone* nicht vollständig trennscharf; deshalb ist eine **explizite Klassifikation der Verbindungen** notwendig. [E1, § 3.9]

Für PyPSA werden zwei abgeleitete Netzkonfigurationen aus derselben kalibrierten Referenz vorgeschlagen:

1. **`LF_EXPANSION`:** interne zonale Ersatzverbindungen werden, soweit dies der dokumentierten Fallklassifikation entspricht, kapazitiv unbeschränkt; ihre **Reaktanzen und KVL bleiben aktiv**. Grenzen, die die Studie als grenzüberschreitend ausweist, bleiben aktiv. Keine Umwandlung interner AC-Verbindungen in unabhängige NTC-`Link`s.
2. **`OPF_CHECK`:** die vier asymmetrischen Sicherheitsgrenzen sind wieder aktiv; für einen gegebenen Ausbaustand können Redispatch- und Engpassfolgen ausgewertet werden.

Für beide Konfigurationen werden dieselbe `busmap`, identische Asset-/Zeitreihenbasis und explizit differenzierte Constraints verwendet. Die **LF-Optimierung** wird nicht als physikalischer Sicherheitsnachweis ausgegeben.

10 Schritt 5: Unabhängige Validierung und wissenschaftliche Abnahme

10.1 Was tatsächlich nachgewiesen werden muss

Zonale Bilanztreue: Gleichung (8) ist ein Strukturtest. Zeitlich prüfbar ist

$$\epsilon_{bal} = \max_t \left\| A_z \Gamma f_t^d - P p_t^d \right\|_\infty . \tag{22}$$

Der Wert muss bis auf das zuvor definierte numerische Toleranzniveau verschwinden. Dasselbe gilt für die exportierte zonale Last-, Dispatch- und Speicherbilanz.

Flussabbildung: Für alle **nicht zur Kalibrierung benutzten** Stunden,

$$\mathrm{RMSE}_e = \sqrt{\frac{\sum_{t \in T_{val}} w_t (\hat{f}_{e,t} - f^\star_{e,t})^2}{\sum_{t \in T_{val}} w_t}}, \qquad \mathrm{MAE}_e = \frac{\sum_t w_t |\hat{f}_{e,t} - f^\star_{e,t}|}{\sum_t w_t}. \tag{23}$$

Zusätzlich sind Maximum und Hochquantile des absoluten Fehlers, getrennt für kritische Stunden, Richtungen, Jahreszeiten und Wetterjahre, auszuweisen. Ein aggregierter globaler RMSE kann hohe Fehler einer einzelnen sicherheitsrelevanten Grenze verdecken.

Sicherheitsäquivalenz: Auswerteraster für jede Grenze und (σ, d): Anzahl Stützpunkte, Belastungsintervall, Güte der Beziehung $R(F)$, Anzahl tatsächlich geprüfter Ausfälle, Verhalten an der ermittelten 100-%-Schwelle, Fehleinstufungen unter einem unabhängigen Kontingenztest und angezeigte Sonderfallbehandlung.

Topologieerhaltung: jede Zone verbunden; Originalknoten genau einmal enthalten; prioritäre kritische Kanten explizit in der Trennkantenliste; keine ungewollte Vermischung von AC-Synchrongebieten über HVDC-Links; keine nicht dokumentiert verschwundenen PST- und Transformatorwirkungen.

10.2 Qualitätsgrenzen: transparent und nicht fälschlich als Norm ausgeben

ENTSO-E illustriert im Anhang qualitative RMSE-Klassen (beispielsweise einen grünen Bereich bei 200 MW beziehungsweise 10 %). Die exakte Verknüpfung absoluter und relativer Grenzwerte sowie die Definition des Bezugsflusses sind nicht vollständig normiert. Diese Abbildung darf nicht unbesehen in einen allgemeinen Produktions-PASS/FAIL-Test umgewandelt werden. [E1, Anhang 6.1]

Eine projektseitige Abnahme könnte deshalb folgende **konfigurierbare** Bedingungen verwenden [I], ohne sie als ENTSO-E-Regeln auszugeben:

Gate	Operationalisierte Prüfung	Aktion bei Verletzung
V1 – Daten	Snapshots und Einspeisungen synchron, Ratings gültig	Abbruch, Datensatz reparieren
V2 – Topologie	Bus-Zonen-Abbildung vollständig und AC-zusammenhängend	Re-Clustering nur betroffener Gebiete
V3 – Erhaltung	Identität (8) und Bilanz (22) numerisch erfüllt	Abbruch, Orientierung/Komponenten korrigieren
V4 – Flussgüte	RMSE/MAE, Tail-Fehler, getrennt nach Wetterfall	lokale Änderung der Partition oder Fitgewichte
V5 – Sicherheit	genügend Stützpunkte und akzeptable N-1-Nachprüfung	Kapazität als **unbestimmt** markieren; keine stille Freigabe
V6 – Zusatzgeräte	PST/HVDC-Effekte verifiziert oder explizit außerhalb Gültigkeit	Erweiterung bzw. enger Gültigkeitsbereich
V7 – Anwendung	LF-/OPF-Modusgrenzen und Asymmetrien korrekt installiert	keine Freigabe für Ausbauoptimierung

Ein nicht kalibrierbarer Ersatzkorridor ist **kein** Anlass, eine scheinpräzise Kapazität zu erfinden. Stattdessen erhält die Ergebnisdatei `status=UNVERIFIED`, den Grund und eine empfohlene Behandlung (z. B. Zonengrenze verbessern oder mehr Daten). Die Entscheidung, solche Fälle in einem LF-Modell temporär unbegrenzt zu behandeln, ist von der OPF-Freigabe strikt zu trennen.

10.3 Validierung an drei verschachtelten Netzeigenschaften

Das Prüfdesign trennt (1) **algebraische** Richtigkeit (Vorzeichen, Einheiten, Bilanzen), (2) **statische Verhaltensähnlichkeit** (interzonale Flüsse und Transfers) und (3) **operative Sicherheit** (N-1-Kontingenzverhalten). Diese Eigenschaften sind logisch nicht austauschbar: Eine exakt erhaltene Bilanz belegt keinen geringen Flussfehler, ein geringer Fluss-RMSE keinen sicheren N-1-Transfer, und ein einzelner sicherer Transferwert keine robuste Sicherheit gegen alle zeitgleichen Änderungen.

11 API-Zuordnung und notwendiger Eigenanteil

Funktion der Reduktion	PyPSA / Standardbibliothek	Zusatzimplementierung	Status
Netz laden/sichern	`pypsa.Network`, `export_to_netcdf`	Daten-Preflight und Versionierung	Direkt + klein
Stündliche DC-Flüsse	`Network.lpf(snapshots=...)`	Einsetzen synchroner Marktdispatchdaten	Direkt + Datenadapter
PTDF	`SubNetwork.calculate_PTDF`	Speicherbewusste Ausschnitte/Orientierung	Direkt
Engpassenergie	`pandas`, `numpy`	Formel (3), Ratings, Ranking	Klein
Elektrische HAC-Busmap	`n.cluster.spatial.busmap_by_hac`	kritische Kanten und Länderbudgets	Direkt + Schutz
Cluster-Netz	`get_clustering_from_busmap`	Transformer-/PST-Erhaltung	Direkt + Pflichtprüfung
Zielgrenzflüsse	`cluster.linemap` als Hilfsindex	orientierte Γ-Matrix auch für Transformer	Klein
Äquivalente Reaktanzen	`scipy.optimize.least_squares`	Residual und sparse DC-Löser gemäß (12)	Eigenmodul
Nullbilanzflüsse	`scipy.sparse`, Zyklenbasis	c^0, KVL-Offset und Tests	Eigenmodul
LODF/BODF	`SubNetwork.calculate_BODF`	Screening für Großnetze	Direkt / skaliert
Saison- und Richtungsgrenzen	`pandas`, `scipy`	N-1-Datenfit (16)–(18)	Eigenmodul
Affine KVL / asymmetrische Grenzen	`n.optimize.create_model`, Linopy	Constraint-Adapter gemäß (14),(20)	Kleiner Integrationsadapter
Netzvergleich	`n.lpf`, `numpy`	Validierungsmetriken und Gate-Report	Klein

Entscheidender Befund: Eine vollständig neue elektrische Netzrepräsentation ist nicht nötig. Der verbleibende fachliche Eigenanteil ist auf **(i) Zonenvalidierung, (ii) Ersatzimpedanz-/No-Load-Identifikation und (iii) N-1-Kapazitätsschätzung** konzentriert. Der Linopy-Adapter nutzt wiederum dieselben Ergebnisse und wird nicht als weiteres Modellierungsframework aufgebaut.

11.1 Einfache Projektstruktur

```
tyndp_pypsa_reduction/
  config.yaml             # Parameter, Datenquellen, Prioritäten und Gates
  pipeline.py             # fünf Orchestrierungsschritte
  zoning_guard.py         # kritische Kanten, Länderbudgets, Zusammenhang
  equivalent_fit.py       # x_eq, c0, Nullbilanz-/KVL-Offset
  n1_capacity.py          # gescreente BODF, Saison-/Richtungswerte
  inputs/                 # versionierte oder referenzierte Eingangsdaten
```

```
  outputs/
    busmap.csv
    linemap.csv
    branch_provenance.csv
    zonal_network.nc
    equivalent_parameters.csv
    directional_seasonal_limits.csv
    validation_metrics.csv
    validation_summary.json
```

Die Dateistruktur ist eine Implementierungsempfehlung. `pipeline.py` steuert lediglich vorhandene PyPSA-Funktionen und drei schmale fachliche Zusatzmodule; es schreibt weder Power-Flow-Algorithmen noch Markt- oder Ausbauoptimierer neu.

11.2 Minimaler ausrufbarer Ablauf

Die folgende Darstellung ist **Architekturcode**, ausdrücklich kein unmittelbar ohne Datenadapter lauffähiges Komplettprogramm:

```python
import pypsa


def reduce_tyndp_case(network_file, cfg):
    n = pypsa.Network(network_file)

    # 1: Eigenfunktion ordnet synchronen Marktdispatch/Assets zu;
    # PyPSA löst anschließend den stündlichen DC-Lastfluss.
    hours = prepare_reference_cases(n, cfg)
    n.lpf(snapshots=hours)
    stress = identify_overload_energy(n, hours, cfg)

    # 2: Eingebaute graphenbewusste HAC plus kritische Kanten.
    busmap = build_checked_busmap(n, stress, cfg)

    # 3: Standard-PyPSA für reine Aggregation; der Zusatzadapter
    # stellt die passive Transformator-/PST-Bilanz kontrolliert her.
    result = n.cluster.spatial.get_clustering_from_busmap(
        busmap, with_time=True,
        aggregate_one_ports={"Load": {}},
        scale_link_capital_costs=False,
    )
    nz, provenance = restore_branch_semantics(n, result, cfg)

    # 4: Mit Zielgrenzflüssen und N-1-Stunden kalibrieren.
    params = fit_equivalent_network(n, nz, busmap, provenance, cfg)
    caps = identify_n1_capacities(n, provenance, stress, cfg)
    apply_parameters_and_constraints(nz, params, caps, cfg)

    # 5: unabhängige Stunden, Engpässe, Ausfälle und Bilanzen.
    report = validate_reduction(n, nz, params, caps, cfg)
    export_reduction(nz, busmap, provenance, params, caps, report, cfg)
    return nz, report
```

Die aufgerufenen Funktionen außer `pypsa.Network`, `n.lpf` und `n.cluster.spatial.get_clustering_from_b` sind **bewusst eigene dünne Projektfunktionen**. Insbesondere kann ein offener Platzhal-

ter für den Linopy-Adapter nicht durch einen scheinbar erfolgreichen `export_to_netcdf()`-Aufruf ersetzt werden: Wenn die affinen KVL-Offsets oder asymmetrischen Grenzen nur in separaten Tabellen liegen, muss die Wiederherstellung des optimierbaren Modells denselben Adapter verpflichtend erneut anwenden. Die NetCDF-Datei allein speichert nicht automatisch beliebige externe Linopy-Constraints.

Für die spätere Kommandozeilennutzung genügt ein einzelner Aufruf einer implementierten Orchestrierung, etwa `python pipeline.py --config config.yaml`. Dieser Aufruf ist ein **vorgeschlagener Projektentrypoint**, kein bereits vorhandenes Tool.

12 Aufwand, Skalierung und bewusste Vereinfachungen

12.1 Drei Strategien im Vergleich

Variante	Entwicklung	Laufzeit/Skalierung	Wissenschaftliche Aussage
A: K-Means/HAC + PyPSA-Standard	sehr gering	günstig bei kleinen bis mittleren Netzen	Baseline, keine N-1-Äquivalenz
B: HAC + Guard + Fit + N-1 (empfohlen)	begrenzte drei Zusatzmodule	Referenzflüsse skalieren mit Stunden; Fit nur über reduziertes Netz	TYNDP-nahe, bei bestandenen Gates
C: komplett neues globales Clustering-/Netzreduktionsmodell	hoch	schwer abschätzbar	Nicht erforderlich, ohne klare Mehrgüte

Die Variante B nutzt die bestehende Struktur und ergänzt genau die methodischen Lücken. Zusätzliche Optimierungstiefe ist nur angebracht, wenn die Validierung dies nachweist. Ein großer europäischer Fall benötigt typischerweise **Daten- und Rechenoptimierung**, aber nicht mehr fachliche Pipeline-Schritte.

12.2 Warum einige vermeintliche Abkürzungen falsch wären

- **Nur geographisches K-Means:** berücksichtigt weder Netzkonnektivität noch Engpasspriorität zuverlässig.
- **Nur `Line.s_nom.sum()`:** kann weder Loop-Flows noch den kritischsten N-1-Fall abbilden.
- **Nur 100 zufällige Stunden:** kann seltene, investitionsrelevante Netzengpässe systematisch übersehen.
- **`s_max_pu=0.7`:** ist eine grobe Sicherheitsmarge, nicht die vier TYNDP-Kapazitäten.
- **HVDC als ungebundener NTC-Link statt Ersatz-AC-Maschenmodell:** verändert die Wirkung von Kirchhoff-Schleifen und ist keine äquivalente AC-Netzreduktion.
- **No-Load-Flüsse nur im Ergebnis nachaddieren:** lässt das verwendete Dispatchmodell physikalisch inkonsistent zur Auswertung werden.
- **Normales `lpf()` als Test des affine-KVL-Modells:** testet die KVL-Offset-Erweiterung nicht.
- **Alle BODF-Kombinationen auf dem vollständigen Detailnetz als dichte Matrix:** kann für kontinentale Netze die Speicherkapazität sprengen.

12.3 Sparsame Rechenstrategie [I]

Empfohlen wird die folgende Aufwandshierarchie:

1. **Detailnetz nur einmal pro konsistenter Topologie faktorisieren**, die relevanten Stunden blockweise auswerten und f_t^d komprimiert speichern.
2. **Nur kritische Leitungen sowie begründete Ausfallkandidaten N-1-screenen**; die globale Komplettmatrix nicht ungeprüft materialisieren. Vollständigkeit der überwachten Kontingenzen bleibt als Scope vermerkt.
3. **Ersatzreaktanzen auf dem kleinen Netz kalibrieren**, wobei die stündlichen Zielflüsse aus dem Detailnetz bereits vorliegen. Warmstarts und sparse Faktorisierungen verwenden.
4. **Stunden zunächst für Fit und unabhängige Prüfung trennen** und nicht erneut nach vermeintlicher Konvergenz die Validierungsstunden in den Fit einmischen.
5. **Nur auffällige Zonengrenzen revidieren** und die Parametrisierung des restlichen Netzes soweit möglich wiederverwenden.

12.4 Grenzen der Nachweisbarkeit

Die beschriebene Architektur kann die **öffentlich dokumentierten Prinzipien** der ENTSO-E methodisch eng nachvollziehen. Sie kann aber weder die nicht veröffentlichten internen Clustering-Gewichte noch den unbekannten Kapazitäts-Fitalgorithmus oder die tatsächlichen TSO-Eingabedatensätze ersetzen. Ohne Originaldaten bedeutet „TYNDP-nah“ **Methodennähe**, nicht Ergebnisidentität. Ein PyPSA-Netzmodell mit gleicher Zonenzahl ist noch keine Reproduktion des TYNDP-Resultats.

13 Schlussfolgerung und konkrete Architekturentscheidung

Architekturentscheidung: Ein einziges PyPSA-`Network` bildet den Daten- und Simulationskern. Die zonale Topologie entsteht aus einer graphenbewussten HAC-Busmap mit explizitem Engpassschutz. Danach ist der integrierte Clustering-Aufruf für die eigentliche räumliche Aggregation zu verwenden. Erst anschließend werden Reaktanzen, strukturelle No-Load-Flüsse und N-1-Kapazitäten gegen die stündlichen Detailnetzflüsse kalibriert; zuletzt folgt eine unabhängige Prüfung.

Diese **fünfstufige Kette** nutzt die stärksten vorhandenen PyPSA-Fähigkeiten und begrenzt die Eigenentwicklung auf drei fachliche Erweiterungen. Die Integration eines gemeinsamen Linopy-Adapters für affine KVL- und richtungsabhängige Grenzbedingungen ist die kleine, aber fachlich entscheidende Anforderung für die anschließende Netzausbauoptimierung. Transformatoren/PST und die Marktdispatch-Provenienz sind harte Preflight-Themen und dürfen nicht auf einen späteren unverbindlichen Prüfpunkt verschoben werden.

Empfohlene erste produktive Freigabe: zunächst BR auf einem vollständig kontrollierten Referenznetz erzeugen, dann TR in genau dieser Reihenfolge qualifizieren: (1) vollständige Komponenten- und Bilanztreue, (2) externe Zielflusskalibrierung, (3) N-1-Kapazitäten, (4) Linopy-Kopplung, (5) unabhängige Stunden- und Kontingenzvalidierung. Wird die geforderte Fluss- oder Sicherheitsqualität nicht erreicht, ist die lokale Busmap zu verbessern; die Modellqualität wird **nicht** durch ungeprüfte zusätzliche Solverkomplexität ersetzt.

14 Literatur und dokumentierte Softwaregrundlagen

[E1] Einzige normative Methodikquelle. ENTSO-E (2026): *TYNDP 2026 Identification of System Needs Methodology. Version for Public Consultation*, 25 February 2026, §§ 3.7–3.9 und Anhang 6.1. https://eepublicdownloads.blob.core.windows.net/public-cdn-container/clean-documents/tyndp-documents/TYNDP2026/TYNDP_2026_identification_of_system_needs_methodology_for_consultation.pdf.

[P1] PyPSA-Anwenderdokumentation, Network Clustering. Dokumentiert eigene `busmap`, K-Means, HAC, Aggregation von Komponenten und Zeitreihen. https://docs.pypsa.org/stable/user-guide/clustering/.

[P2] PyPSA-API, SpatialClusteringAccessor. Signaturen für `busmap_by_hac`, `get_clustering_from_busm` `aggregate_one_ports`, `custom_line_groupers` und verwandte Argumente. https://docs.pypsa.org/latest/api/networks/cluster/.

[P3] PyPSA-API, SubNetwork. Signaturen und dense-array-Semantik für `calculate_PTDF` und `calculate_BODF`. https://docs.pypsa.org/latest/api/networks/subnetwork/.

[P4] PyPSA-Anwenderdokumentation, Optimization. Unterstützung der programmgesteuerten Erstellung und Modifikation eines Linopy-Modells und der KVL-Bedingungen. https://docs.pypsa.org/latest/user-guide/network-optimization/.

[P5] PyPSA-Anwenderdokumentation, Optimisation Power Flow und Transformer. KVL-Zyklenformulierung, NTC-Link-Abgrenzung, Phasenverschiebung und Reaktanzdarstellung. https://docs.pypsa.org/latest/user-guide/optimization/power-flow/ und https://docs.pypsa.org/latest/user-guide/components/transformers/.

[P6] PyPSA-Quellcode, `pypsa/clustering/spatial.py`. Aktuelle Struktur von `get_clustering_from_busm` vordefinierte Leitungsstrategien und Komponentenweitergabe, abgerufen am 16.10.2026. https://github.com/PyPSA/PyPSA/blob/master/pypsa/clustering/spatial.py. Für produktive Reproduzierbarkeit ist **eine konkrete Git-Revision** statt des beweglichen Zweigs `master` zu pinnen.

Quellenregel: [P1–P6] werden nur zur Beurteilung der PyPSA-Werkzeugfähigkeit verwendet, **nicht** als zusätzliche ENTSO-E-Methodikvorgabe. Sämtliche mathematischen Ergänzungen werden ausdrücklich als [R] oder [I] gekennzeichnet.